

Harmonizing datums and tectonic epochs in drone data using PROJ



Wednesday, Sep. 11, 2024, 10:30am - 11:00am
Grand C
FOSS4GNA 2024 St. Louis

Dan Mahr
Lead Software Engineer @ DroneDeploy

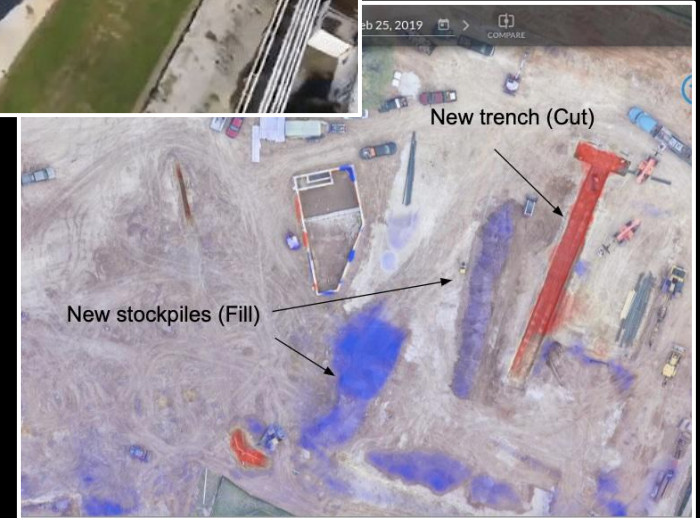


DroneDeploy

Maximum spatial resolution:
1 cm/pixel (zoom level 24).

Accuracy of regular GPS
sensors: 100 to 500 cm.

Misalignment of consecutive
maps inhibits analyses e.g.
change detection, cut/fill
comparison, asset tracking.



High accuracy via Ground Control Points (GCPs)

**Survey points on the ground,
photogrammetry fits map to coordinates.**

👍 **Pros: most accurate, works anywhere**

👎 **Cons: labor intensive, error prone**



High accuracy via RTK, PPK

Correct raw GPS signals during flight (RTK) or after flight (PPK).

👍 **Pros: easier, simpler for drone pilot**

👎 **Cons: sometimes less accurate than GCPs, limited geographic availability**



Continuously Operating Reference Station (CORS) "[MOBF](#)" in Bellefontaine, MO

High accuracy results



Misalignment : why?



GCPs vs PPK/RTK

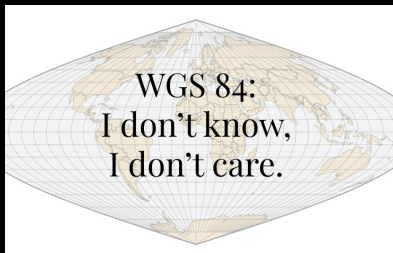


**PPK/RTK vs. PPK/RTK,
years apart**

Datums : static and dynamic

Not every longitude,
latitude coordinate
pair is in WGS84!

See [this FOSS4GE 2024 talk](#)
from PROJ maintainer Javier
Jimenez Shaw



Example: MODOT
CORS "[MOBF](#)" in
Bellefontaine, MO



Longitude: -90.24658117
Latitude: 38.76431088

NAD83(2011)

"static" datum: tied to
North American tectonic plate

109 cm
difference!

Longitude: -90.24658986
Latitude: 38.76431797

ITRF2014

"dynamic" datum: not tied
to any tectonic plate, must
know survey date (epoch)

Misalignment: understood!



**GCPs surveyed in a static datum
won't align to PPK/RTK captured
in a dynamic datum**



**PPK/RTK captured in a dynamic
datum won't align to PPK/RTK
captured years before/after**

Epoch : when the data was surveyed

Must know epoch – *when* data was surveyed
– to achieve cm-level accuracy with dynamic datums e.g. WGS84, ITRF.

ITRF2014@2024.698

Epoch : when the data was surveyed

Must know epoch – *when* data was surveyed
– to achieve cm-level accuracy with dynamic datums e.g. WGS84, ITRF.

Kinematic transformations simulate continental drift to reconcile data from different epochs of the same dynamic datum.

Epoch is typically not needed for static datums e.g. NAD83.

International Terrestrial Reference Frame

2014 edition/version (*not the epoch!*)

ITRF2014@2024.698

Decimal year: 2024, $0.698 * 366 = 255\text{th day}$

"Datum Harmonization"

🤔 Doesn't align!



Images + GCPs in
NAD83

PROJ kinematic transformation from
NAD83@2010 to ITRF2014@2010

PPK/RTK images in
ITRF2014@Mission

PROJ kinematic transformation from
ITRF2014@Mission to ITRF2014@2010

GIS/CAD drawings
in NAD83

PROJ kinematic transformation from
NAD83@2010 to ITRF2014@2010

🎉 Does align! 🎉

Images + GCPs in
ITRF2014@2010

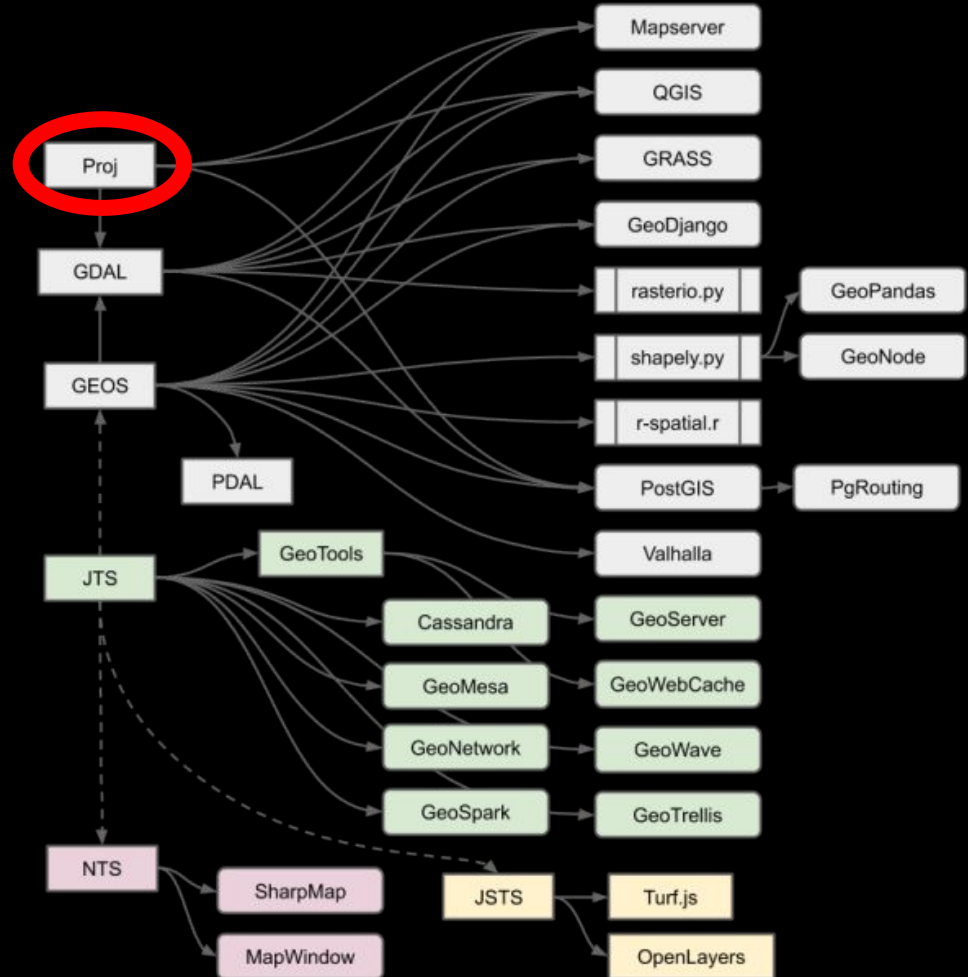
PPK/RTK images in
ITRF2014@2010

GIS/CAD drawings
in ITRF2014@2010

Part II: Implementation Details

PROJ

- Open-source coordinate transformation library, proj.org.
- Underpins much of the FOSS4G ecosystem, including QGIS and GDAL.



PROJ: axis order

Dan Mahr Oct 21, 2023

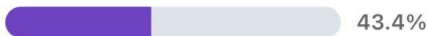
Which axis ordering is best?

69 responses

Longitude/Latitude



Latitude/Longitude



Some EPSG codes officially use y,x instead of x,y ordering. See the [FAQ: "Why is the axis ordering in PROJ not consistent?"](#)

DroneDeploy uses x,y everywhere.

The `always_xy=True` kwarg to [PyProj transformers](#) is the way.

Warning

The axis order may be swapped if the source and destination CRS's are defined as having the first coordinate component point in a northerly direction (See PROJ FAQ on [axis order](#)). You can check the axis order with the `pyproj.crs.CRS` class. If you prefer to keep your axis order as always x,y, you can use the `always_xy` option when creating the `pyproj.transformer.Transformer`.

PROJ: transformation methods

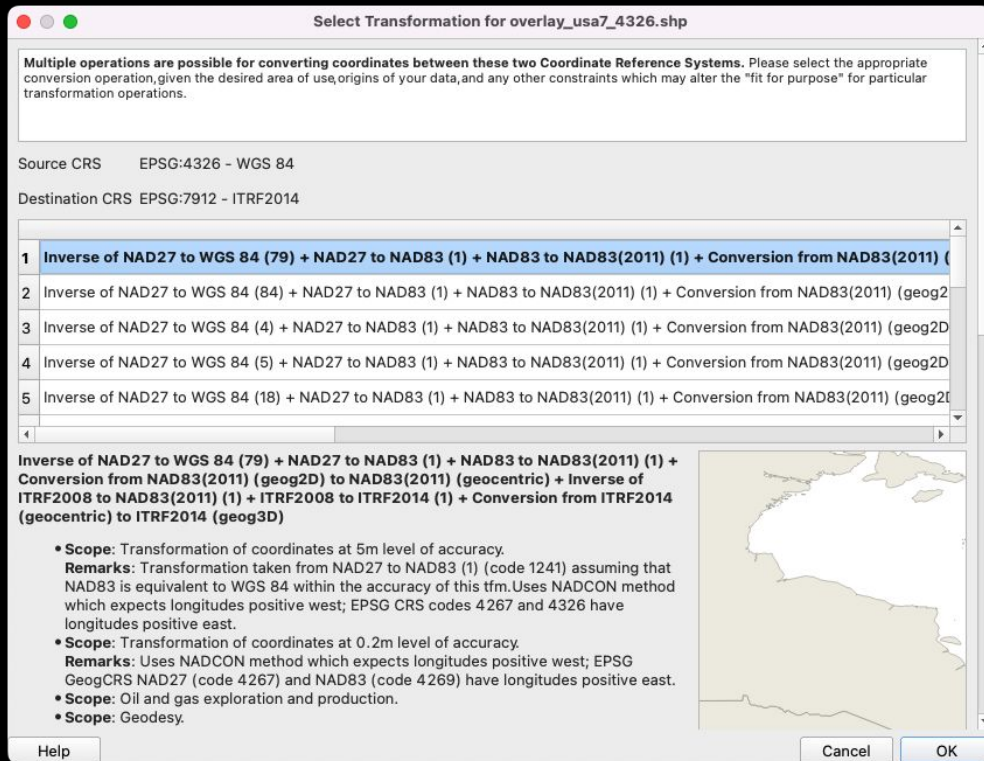
Multiple methods may exist for transforming between two coordinate systems.

QGIS prompts for the method to use

Command line tools (e.g. gdalwarp) just pick the first one.

Use [projinfo](#) to list all:

```
projinfo -s epsg:4283  
-t epsg:7912 -o PROJ
```

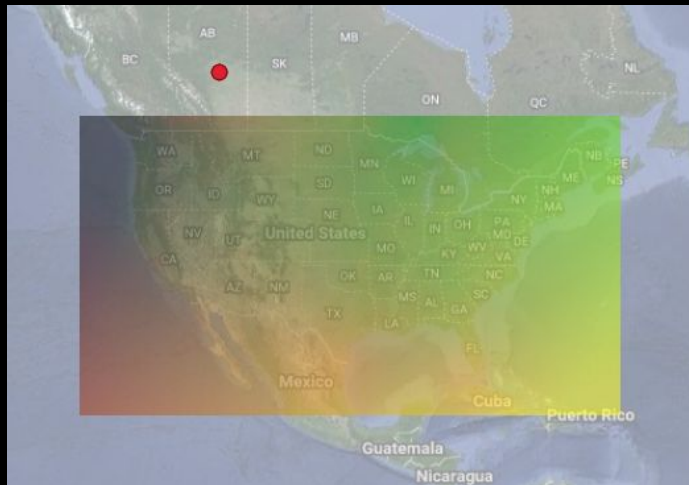


PROJ: transformation grid files

Most transformation methods only require a few parameters.

Some transformations use [grid files](#) for improved accuracy. There are many ways to get these:

- Use [PROJ Docker image](#)
- Download all using [projsync](#)
- Stream from [PROJ CDN](#)
 - PROJ_NETWORK=ON env var.
 - PyProj: [set_network_enabled\(\)](#)



PROJ.org Datumgrid CDN

The `s3://cdn.proj.org` bucket is hosted by the [Amazon Public Datasets](#) program, and it can be accessed via HTTP/S at cdn.proj.org. CDN services are provided by the AWS Public Dataset team via [CloudFront](#).

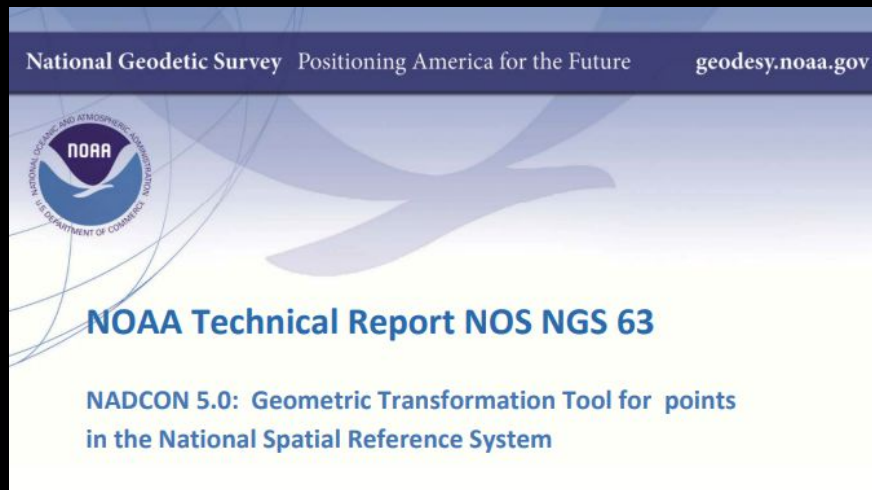
The CDN contains [Cloud Optimized GeoTIFF](#) datasets which are mirrored and managed by the <https://github.com/OSGeo/PROJ-data/> GitHub project. Files in the CDN are designed to be used by [PROJ 7](#) or later, but any software project wishing to use the CDN for shifting support are encouraged to participate in the project and leverage the CDN.

PROJ: versions

Recommend PROJ 9.2+ with [gridshift method](#) for [NADCON5](#) grid files.

Beware when upgrading: number and ordering of transformation methods is not a stable part of PROJ API.

At DroneDeploy, new GCP maps must align with old GCP maps.



PROJ: pipelines

At DroneDeploy, we hardcode transformation methods from every CRS to/from WGS84/ITRF2014 to ensure stability over time.

Transformation methods are serialized as [PROJ pipelines](#).

Pipelines are passed to PROJ, overriding its choice of method.

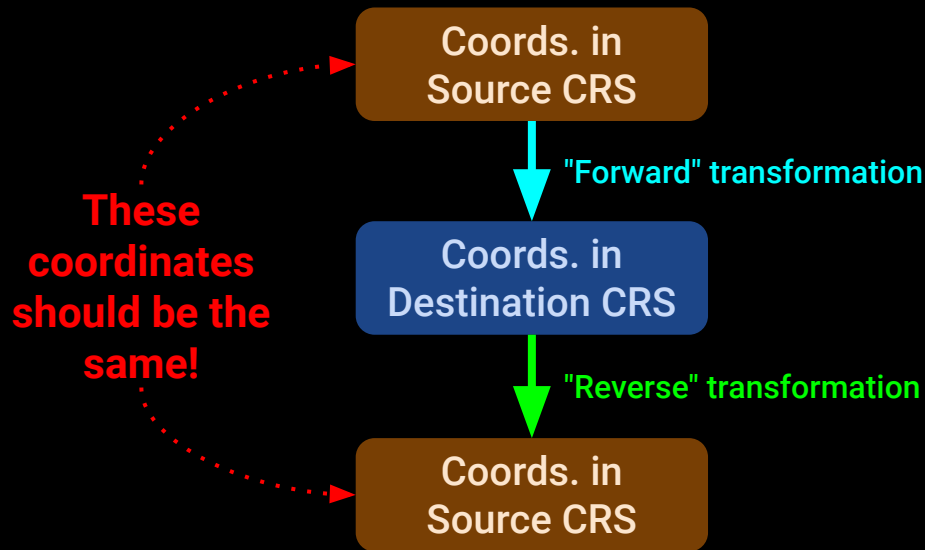
- [gdalwarp -ct](#)
- [ogr2ogr -ct](#)
- [pyproj.Transformer.from_pipeline\(\)](#)
- [PDAL projpipeline filter](#)

```
proj=pipeline
step proj=unitconvert xy_in=deg xy_out=rad
step proj=hgridshift
    grids=au_icsm_GDA94_GDA2020_conformal_and_d
    istortion.tif
step proj=cart ellps=GRS80
step inv proj=helmert x=0 y=0 z=0 rx=0 ry=0
    rz=0 s=0 dx=0 dy=0 dz=0 drx=0.00150379
    dry=0.00118346 drz=0.00120716 ds=0
    t_epoch=2020 convention=coordinate_frame
step inv proj=cart ellps=GRS80
step proj=unitconvert xy_in=rad z_in=m
    xy_out=deg z_out=m
```

PROJ: round-trip validation

Double check that your **reverse transformation** truly is the reverse of the **forward transformation**.

Occasionally, PROJ returns a weird ordering and coordinates won't round-trip, which is bad.



PROJ: epoch transformer

```
t = pyproj.Transformer.from_pipeline("""
    proj=pipeline ellps=GRS80
    step proj=unitconvert xy_in=deg xy_out=rad
    step proj=cart
    step inv init=ITRF2014:{plate}
        t_epoch={dst_epoch}
    step inv proj=cart
    step proj=unitconvert xy_in=rad xy_out=deg
    """)
t.transform(x, y, z, src_epoch)
```

PROJ pipeline for transforming ITRF2014 coordinates from {src_epoch} to {dst_epoch} using one of the [11 available](#) {plate} codes.

This uses [PROJ init files](#) shorthand.



Plate boundaries [from fraxen on Github](#).

PROJ: epoch transformer

```
t = pyproj.Transformer.from_pipeline("""
    proj=pipeline ellps=GRS80
    step proj=unitconvert xy_in=deg xy_out=rad
    step proj=cart
    step inv init=ITRF2014:NOAM
        t_epoch=2024.0
    step inv proj=cart
    step proj=unitconvert xy_in=rad xy_out=deg
    """)
lng1, lat1 = -90.24658117, 38.76431088
lng2, lat2, *_ = t.transform(lng1, lat1,
                             0, 2023)

geod = pyproj.geod.Geod(ellps='WGS84')
print(geod.inv(lng1, lat1, lng2, lat2)[2])

0.014906229324212157
```

PROJ pipeline for transforming ITRF2014 coordinates from {src_epoch} to {dst_epoch} using one of the [11 available](#) {plate} codes.

This uses [PROJ init files](#) shorthand.



Plate boundaries [from fraxen on Github](#).

PROJ: specifying source epoch

Theoretically possible:

- `gdalwarp -s_coord_epoch`
- `ogr2ogr -s_coord_epoch`

But these didn't always work in our testing.

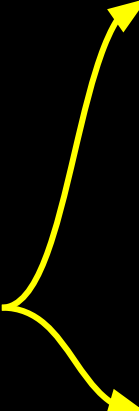
Also, PDAL has no such option.

Workaround is to **set the source epoch inside the PROJ pipeline itself via the set operator**. Needed at both start and end for [reasons](#).

This overrides any `-s_coord_epoch` passed to the transformer.

This also fixes NaN results when passing a null epoch to pipelines with deformation models e.g. NZGD2000.

```
proj=pipeline ellps=GRS80
step proj=set v_4={src_epoch}
step proj=unitconvert xy_in=deg
xy_out=rad
step proj=cart
step inv
init=ITRF2014:{plate}
t_epoch={dst_epoch}
step inv proj=cart
step proj=unitconvert xy_in=rad
xy_out=deg
step proj=set v_4={src_epoch}
```



PROJ: elevation unit conversion

We want to preserve elevation values. The only change should be unit conversion.

PROJ doesn't convert elevation units unless there an explicit vertical datum.

Transforming to/from ITRF2014 often does Helmert transform, changing elevations.

The solution:

1. Convert feet to meters **first**
2. Push the z-coordinate **before** the Helmert and pop it **after**.

```
proj=pipeline
step proj=unitconvert
  xy_in=us-ft xy_out=m z_in=us-ft z_out=m
step proj=push v_3
step inv proj=lcc
  lat_0=41 lon_0=-71.5
  lat_1=42.683333333333 lat_2=41.716666666667
  x_0=200000.0001016 y_0=750000 ellps=GRS80
step proj=cart ellps=GRS80
step inv proj=helmert
  x=1.0053 y=-1.9092 z=-0.5416
  rx=0.0267814 ry=-0.0004203 rz=0.0109321
  dx=0.0008 dy=-0.0006 dz=-0.0014
  drx=6.67e-05 dry=-0.0007574 drz=-5.13e-05
  s=0.00037 ds=-7e-05 t_epoch=2010
  convention=coordinate_frame
step inv proj=cart ellps=GRS80
step proj=pop v_3
step proj=unitconvert
  xy_in=rad z_in=m xy_out=deg z_out=m
```

Summary

- Not every lng,lat is WGS84!
- Static datums (e.g. NAD83, GDA94, NZGD2000) move with a tectonic plate.
- Dynamic datums (e.g. WGS84, ITRF2014) are not tied to a tectonic plate.
- The coordinates of an unmoving monument in a dynamic datum change over time.
- RTK/PPK data is often in a dynamic datum whose epoch is the capture date.
- The epoch (survey date) is needed for high accuracy with dynamic datums.
- PROJ can harmonize data from different datums and epochs.

PROJ best practices for datum harmonization:

- `always_xy=True` disambiguates axis order.
- Use PROJ 9.2+ for NADCON5 support.
- Multiple transformation methods may exist between a given pair of coordinate systems.
- `PROJ_NETWORK=ON` streams grid files as needed.
- Hardcode PROJ pipelines to ensure stability of transformation methods across PROJ versions.
- Use plate motion models to transform ITRF2014 data from one epoch to another.
- Use `set` operator to override source epoch.
- Use `unitconvert`, `push`, and `pop` operators to handle non-meter elevation units.

https://github.com/dmahr1/talks/tree/main/2024_FOSS4GNA